

```

1 // Comments marked by initials ER, KP, KL, DS, EC
2 //#####
3 // FILE:    FinalProjectStarter_main.c
4 //
5 // TITLE:   Final Project Starter
6 //#####
7
8 // Included Files
9 #include <stdio.h>
10 #include <stdlib.h>
11 #include <stdarg.h>
12 #include <string.h>
13 #include <math.h>
14 #include <limits.h>
15 #include "F28x_Project.h"
16 #include "F2837xD_SWPrioritizedIsrLevels.h"
17 #include "driverLib.h"
18 #include "device.h"
19 #include "F28379dSerial.h"
20 #include "song.h"
21 #include "dsp.h"
22 #include "fpu32/fpu_rfft.h"
23 #include "xy.h"
24 #include "MatrixMath.h"
25 #include "SE423Lib.h"
26 #include "OptiTrack.h"
27
28 #define PI          3.1415926535897932384626433832795
29 #define TWOPI       6.283185307179586476925286766559
30 #define HALFPI      1.5707963267948966192313216916398
31 // The Launchpad's CPU Frequency set to 200 you should not change this value
32 #define LAUNCHPAD_CPU_FREQUENCY 200
33
34 // ----- code for CAN start here -----
35 #include "F28379dCAN.h"
36 // #define TX_MSG_DATA_LENGTH      4
37 // #define TX_MSG_OBJ_ID           0 //transmit
38
39 #define RX_MSG_DATA_LENGTH      8
40 #define RX_MSG_OBJ_ID_1         1 //measurement from sensor 1
41 #define RX_MSG_OBJ_ID_2         2 //measurement from sensor 2
42 #define RX_MSG_OBJ_ID_3         3 //quality from sensor 1
43 #define RX_MSG_OBJ_ID_4         4 //quality from sensor 2
44 // ----- code for CAN end here -----
45
46 // Interrupt Service Routines predefinition
47 __interrupt void cpu_timer0_isr(void);
48 __interrupt void cpu_timer1_isr(void);
49 __interrupt void cpu_timer2_isr(void);
50 __interrupt void SWI1_HighestPriority(void);
51 __interrupt void SWI2_MiddlePriority(void);
52 __interrupt void SWI3_LowestPriority(void);
53 __interrupt void ADCC_ISR(void);
54 __interrupt void SPIB_isr(void);
55 // ----- code for CAN start here -----
56 __interrupt void can_isr(void);
57 // ----- code for CAN end here -----
58
59 void setF28027EPWM1A(float controleffort);
60 int16_t EPwm1A_F28027 = 1500;
61 void setF28027EPWM2A(float controleffort);
62 int16_t EPwm2A_F28027 = 1500;
63
64
65 // ----- code for CAN start here -----
66 // volatile uint32_t txMsgCount = 0;
67 // extern uint16_t txMsgData[4];
68
69 volatile uint32_t rxMsgCount_1 = 0;
70 volatile uint32_t rxMsgCount_2 = 0;
71 extern uint16_t rxMsgData[8];
72
73 uint32_t dis_raw_1[2];
74 uint32_t dis_raw_2[2];
75 uint32_t dis_1 = 0;
76 uint32_t dis_2 = 0;
77
78 uint32_t quality_raw_1[4];
79 uint32_t quality_raw_2[4];
80 float quality_1 = 0.0;
81 float quality_2 = 0.0;
82
83 uint32_t lightlevel_raw_1[4];
84 uint32_t lightlevel_raw_2[4];
85 float lightlevel_1 = 0.0;
86 float lightlevel_2 = 0.0;
87
88 uint32_t measure_status_1 = 0;
89 uint32_t measure_status_2 = 0;
90
91 volatile uint32_t errorFlag = 0;
92 // ----- code for CAN end here -----
93
94 uint32_t numTimer0calls = 0;
95 uint16_t UARTPrint = 0;
96
97 float printLV1 = 0;
98 float printLV2 = 0;
99
100 float printLinux1 = 0;
101 float printLinux2 = 0;
102
103 uint16_t LADARi = 0;
104 char G_command[] = "G04472503\n"; //command for getting distance -120 to 120 degree
105 uint16_t G_len = 11; //length of command
106 xy ladar_pts[228]; //xy data
107 float LADARrightfront = 0;
108 float LADARfront = 0;
109 float LADARtemp_x = 0;
110 float LADARtemp_y = 0;
111 extern datapts ladar_data[228];
112
113 extern uint16_t newLinuxCommands;
114 extern float LinuxCommands[CMDNUM_FROM_FLOATS];
115
116 extern uint16_t NewLVData;
117 extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
118 extern LVSendFloats_t DataToLabView;
119 extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
120 extern uint16_t LADARpingpong;
121 extern uint16_t NewCAMdataThresho1d; // Flag new data
122 extern float fromCAMvaluesThresho1d[CAMNUM_FROM_FLOATS];
123 extern uint16_t NewCAMdataThresho1d2; // Flag new data
124 extern float fromCAMvaluesThresho1d2[CAMNUM_FROM_FLOATS];
125
126 extern uint16_t NewCAMdataAprilTag1; // Flag new data
127 extern float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
128
129 float tagid = 0;
130 float tagx = 0;
131 float tagy = 0;
132 float tagz = 0;
133 float tagthetax = 0;
134 float tagthetay = 0;
135 float tagthetaz = 0;
136 int32_t numtag = 0;
137 int32_t AprilTagState = 10;
138 float KpAprilTag = -1.0;
139 float AprilTagSetPoint = 0;
140 int32_t AprilTagCount = 0;

```

```

141
142 float MaxAreaThreshold1 = 0;
143 float MaxColThreshold1 = 0;
144 float MaxRowThreshold1 = 0;
145 float NextLargestAreaThreshold1 = 0;
146 float NextLargestColThreshold1 = 0;
147 float NextLargestRowThreshold1 = 0;
148 float NextNextLargestAreaThreshold1 = 0;
149 float NextNextLargestColThreshold1 = 0;
150 float NextNextLargestRowThreshold1 = 0;
151
152 float MaxAreaThreshold2 = 0;
153 float MaxColThreshold2 = 0;
154 float MaxRowThreshold2 = 0;
155 float NextLargestAreaThreshold2 = 0;
156 float NextLargestColThreshold2 = 0;
157 float NextLargestRowThreshold2 = 0;
158 float NextNextLargestAreaThreshold2 = 0;
159 float NextNextLargestColThreshold2 = 0;
160 float NextNextLargestRowThreshold2 = 0;
161
162 uint32_t numThres1 = 0;
163 uint32_t numThres2 = 0;
164
165 pose ROBOTps = {0,0,0}; //robot position
166 pose LADARps = {3.5/12.0,0,1}; // 3.5/12 for front mounting, theta is not used in this current code
167 float LADARxoffset = 0;
168 float LADARyoffset = 0;
169
170 uint32_t timecount = 0;
171 int16_t RobotState = 1;
172 int16_t checkfronttally = 0;
173 int32_t WallFollowtime = 0;
174
175 #define NUMWAYPOINTS 10 //Modified NUMWAYPOINTS from 8 to 10 for two extra waypoint targets for the robot path - DS
176 uint16_t statePos = 0;
177 pose robotdest[NUMWAYPOINTS]; // array of waypoints for the robot
178 uint16_t i = 0;//for loop
179
180 uint16_t right_wall_follow_state = 2; // right follow
181 float Kp_front_wall = -2.0;
182 float front_turn_velocity = 0.2;
183 float left_turn_Stop_threshold = 3.5;
184 float Kp_right_wal = -4.0;
185 float ref_right_wall = 1.1;
186 float foward_velocity = 1.0;
187 float left_turn_Start_threshold = 1.3;
188 float turn_saturation = 2.5;
189
190 //RC Servo
191 float Gate_0 = -67.74;
192 float Gate_C = 0.40;
193 float RTong = -49.34; //KLEC: ball goes to the left
194 float MTong = -11.49;
195 float LTong = 1.47; //KLEC: ball goes to the right
196
197 float x_pred[3][1] = {{0},{0},{0}}; // predicted state
198
199 //more kalman vars
200 float B[3][2] = {{1,0},{1,0},{0,1}}; // control input model
201 float u[2][1] = {{0},{0}}; // control input in terms of velocity and angular velocity
202 float Bu[3][1] = {{0},{0},{0}}; // matrix multiplication of B and u
203 float z[3][1]; // state measurement
204 float eye3[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // 3x3 identity matrix
205 float K[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // optimal Kalman gain
206 #define ProcUncert 0.0001
207 #define CovScalar 10
208 float Q[3][3] = {{ProcUncert,0,ProcUncert/CovScalar},
209 {0,ProcUncert,ProcUncert/CovScalar},
210 {ProcUncert/CovScalar,ProcUncert/CovScalar,ProcUncert}}; // process noise (covariance of encoders and gyro)
211 #define MeasUncert 1
212 float R[3][3] = {{MeasUncert,0,MeasUncert/CovScalar},
213 {0,MeasUncert,MeasUncert/CovScalar},
214 {MeasUncert/CovScalar,MeasUncert/CovScalar,MeasUncert}}; // measurement noise (covariance of OptiTrack Motion Capture measurement)
215 float S[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance
216 float S_inv[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance matrix inverse
217 float P_pred[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // predicted covariance (measure of uncertainty for current position)
218 float temp_3x3[3][3]; // intermediate storage
219 float temp_3x1[3][1]; // intermediate storage
220 float ytilde[3][1]; // difference between predictions
221
222 // Optitrack Variables
223 int32_t OptiNumUpdatesProcessed = 0;
224 pose OPTITRACKps;
225 extern uint16_t new_optitrack;
226 extern float Optitrackdata[OPTITRACKDATASIZE];
227 int16_t newOPTITRACKpose=0;
228
229 int16_t adcc2result = 0;
230 int16_t adcc3result = 0;
231 int16_t adcc4result = 0;
232 int16_t adcc5result = 0;
233 float adcc2Volt = 0.0;
234 float adcc3Volt = 0.0;
235 float adcc4Volt = 0.0;
236 float adcc5Volt = 0.0;
237 int32_t numADCCalls = 0;
238 float LeftWheel = 0;
239 float RightWheel = 0;
240 float LeftWheel_1 = 0;
241 float RightWheel_1 = 0;
242 float LeftVel = 0;
243 float RightVel = 0;
244 float uLeft = 0;
245 float uRight = 0;
246 float HandValue = 0;
247 float vref = 0;
248 float turn = 0;
249 float gyro9250_drift = 0;
250 float gyro9250_angle = 0;
251 float old_gyro9250 = 0;
252 float gyroLPR510_angle = 0;
253 float gyroLPR510_offset = 0;
254 float gyroLPR510_drift = 0;
255 float old_gyroLPR510 = 0;
256 float gyro9250_radians = 0;
257 float gyroLPR510_radians = 0;
258
259 int16_t readdata[25];
260 int16_t IMU_data[9];
261
262 float accelx = 0;
263 float accely = 0;
264 float accelz = 0;
265 float gyrox = 0;
266 float gyroy = 0;
267 float gyroz = 0;
268
269 // Needed global Variables
270 float accelx_offset = 0;
271 float accely_offset = 0;
272 float accelz_offset = 0;
273 float gyrox_offset = 0;
274 float gyroy_offset = 0;
275 float gyroz_offset = 0;
276 int16_t calibration_state = 0;
277 int32_t calibration_count = 0;
278 int16_t doneCal = 0;
279
280 #define MPU9250 1
281 #define DAN28027 2
282 int16_t CurrentChip = MPU9250;

```

```

283 int16_t DAN28027Garbage = 0;
284 int16_t dan28027adc1 = 0;
285 int16_t dan28027adc2 = 0;
286 uint16_t MPU925IgnoreCNT = 0; //This is ignoring the first few interrupts if ADCC_ISR and start sending to IMU after these first few interrupts.
287 //Added variables to store camera-based ball distances - DS
288 float robotToBall2 = 0;
289 float robotToBall1 = 0;
290
291 // angle of the hand encoder - ER
292 float RCangle = 0.0;
293 float colcentroid1 = 0.0;
294 float colcentroid2 = 0.0;
295 float kpvision = -0.07; // vision proportional gain chosen so robot turns toward the ball centroid without overcorrecting - DS
296
297 int16_t count = 0;
298 int16_t statecount = 2000;
299 int16_t dwell=0;
300 float current= 0.0;
301 float target = 0.0;
302
303 void main(void)
304 {
305     // PLL, WatchDog, enable Peripheral Clocks
306     // This example function is found in the F2837xD_SysCtrl.c file.
307     InitSysCtrl();
308
309     InitGpio();
310
311     InitSE423DefaultGPIO();
312
313     //Scope for Timing
314     GPIO_SetupPinMux(11, GPIO_MUX_CPU1, 0);
315     GPIO_SetupPinOptions(11, GPIO_OUTPUT, GPIO_PUSH_PULL);
316     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
317
318     GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
319     GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSH_PULL);
320     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
321
322     GPIO_SetupPinMux(52, GPIO_MUX_CPU1, 0);
323     GPIO_SetupPinOptions(52, GPIO_OUTPUT, GPIO_PUSH_PULL);
324     GpioDataRegs.GPBCLEAR.bit.GPIO52 = 1;
325
326     GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
327     GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSH_PULL);
328     GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;
329
330     GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
331     GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSH_PULL);
332     GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
333
334     GPIO_SetupPinMux(67, GPIO_MUX_CPU1, 0);
335     GPIO_SetupPinOptions(67, GPIO_OUTPUT, GPIO_PUSH_PULL);
336     GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1;
337
338     // ----- code for CAN start here -----
339     //GPIO17 - CANRXB
340     GPIO_SetupPinMux(17, GPIO_MUX_CPU1, 2);
341     GPIO_SetupPinOptions(17, GPIO_INPUT, GPIO_ASYNC);
342
343     //GPIO12 - CANTXB
344     GPIO_SetupPinMux(12, GPIO_MUX_CPU1, 2);
345     GPIO_SetupPinOptions(12, GPIO_OUTPUT, GPIO_PUSH_PULL);
346     // ----- code for CAN end here -----
347
348
349
350     // ----- code for CAN start here -----
351     // Initialize the CAN controller
352     InitCANB();
353
354     // Set up the CAN bus bit rate to 1000 kbps
355     setCANBitRate(200000000, 1000000);
356
357     // Enables Interrupt line 0, Error & Status Change interrupts in CAN_CTL register.
358     CanbRegs.CAN_CTL.bit.IE0= 1;
359     CanbRegs.CAN_CTL.bit.EIE= 1;
360     // ----- code for CAN end here -----
361
362     // Clear all interrupts and initialize PIE vector table:
363     // Disable CPU interrupts
364     DINT;
365
366     // Initialize the PIE control registers to their default state.
367     // The default state is all PIE interrupts disabled and flags
368     // are cleared.
369     // This function is found in the F2837xD_PieCtrl.c file.
370     InitPieCtrl();
371
372     // Disable CPU interrupts and clear all CPU interrupt flags:
373     IER = 0x0000;
374     IFR = 0x0000;
375
376     // Initialize the PIE vector table with pointers to the shell Interrupt
377     // Service Routines (ISR).
378     // This will populate the entire table, even if the interrupt
379     // is not used in this example. This is useful for debug purposes.
380     // The shell ISR routines are found in F2837xD_DefaultIsr.c.
381     // This function is found in F2837xD_PieVect.c.
382     InitPieVectTable();
383
384     // Interrupts that are used in this example are re-mapped to
385     // ISR functions found within this project
386     EALLOW; // This is needed to write to EALLOW protected registers
387     PieVectTable.TIMER0_INT = &cpu_timer0_isr;
388     PieVectTable.TIMER1_INT = &cpu_timer1_isr;
389     PieVectTable.TIMER2_INT = &cpu_timer2_isr;
390     PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
391     PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
392     PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
393     PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
394     PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
395     PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
396     PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
397     PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
398     PieVectTable.ADCC1_INT = &ADCC_ISR;
399     PieVectTable.SPIB_RX_INT = &SPIB_isr;
400     PieVectTable.EMIF_ERROR_INT = &SWI1_HighestPriority; // Using Interrupt12 interrupts that are not used as SWIs
401     PieVectTable.RAM_CORRECTABLE_ERROR_INT = &SWI2_MiddlePriority;
402     PieVectTable.FLASH_CORRECTABLE_ERROR_INT = &SWI3_LowestPriority;
403     // ----- code for CAN start here -----
404     PieVectTable.CANB0_INT = &can_isr;
405     // ----- code for CAN end here -----
406     EDIS; // This is needed to disable write to EALLOW protected registers
407
408     // Initialize the CpuTimers Device Peripheral. This function can be
409     // found in F2837xD_CpuTimers.c
410     InitCpuTimers();
411
412     // Configure CPU-Timer 0, 1, and 2 to interrupt every second:
413     // 200MHz CPU Freq, Period (in uSeconds)
414     ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000); // Currently not used for any purpose
415     ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000); // !!!! Important, Used to command LADAR every 100ms. Do not Change.
416     ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000); // Currently not used for any purpose
417
418     // Enable CpuTimer Interrupt bit TIE
419     CpuTimer0Regs.TCR.all = 0x4000;
420     CpuTimer1Regs.TCR.all = 0x4000;
421     CpuTimer2Regs.TCR.all = 0x4000;
422
423     DELAY_US(1000000); // Delay 1 second giving Lidar Time to power on after system power on
424

```

```

425 init_serialSCIB(&SerialB,19200);
426 init_serialSCIC(&SerialC,19200);
427
428 for (LADARi = 0; LADARi < 228; LADARi++) {
429     ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329; //0.017453292519943 is pi/180, multiplication is faster; 0.3515625 is 360/1024
430 }
431
432 init_eQEPs();
433 init_EPWM1and2();
434 init_RCServoPWM_3AB_5AB_6A();
435 init_ADCsAndDACs();
436 setupSpib();
437
438 EALLOW;
439 EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOC on A group
440 EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
441 EPwm4Regs.ETSEL.bit.SOCASEL = 2; // Select Event when counter equal to PRD
442 EPwm4Regs.ETPS.bit.SOCAPRD = 1; // Generate pulse on 1st event
443 EPwm4Regs.TBCTR = 0x0; // Clear counter
444 EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
445 EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
446 EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
447 EPwm4Regs.TBPRD = 50000; // Set Period to 1ms sample. Input clock is 50MHz.
448 // Notice here that we are not setting CMPA or CMPB because we are not using the PWM signal
449 EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
450 //EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, wait to do this right before enabling interrupts
451 EDIS;
452
453 // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
454 // which is connected to CPU-Timer 1, and CPU int 14, which is connected
455 // to CPU-Timer 2: int 12 is for the SWI.
456 IER |= M_INT1;
457 IER |= M_INT6;
458 IER |= M_INT8; // SCIC SCID
459 IER |= M_INT9; // SCIA
460 IER |= M_INT12;
461 IER |= M_INT13;
462 IER |= M_INT14;
463
464 // Enable TINT0 in the PIE: Group 1 interrupt 7
465 PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
466 PieCtrlRegs.PIEIER1.bit.INTx3 = 1;
467 PieCtrlRegs.PIEIER6.bit.INTx3 = 1;
468 PieCtrlRegs.PIEIER12.bit.INTx9 = 1; //SWI1
469 PieCtrlRegs.PIEIER12.bit.INTx10 = 1; //SWI2
470 PieCtrlRegs.PIEIER12.bit.INTx11 = 1; //SWI3 Lowest priority
471 // ----- code for CAN start here -----
472 // Enable CANB in the PIE: Group 9 interrupt 7
473 PieCtrlRegs.PIEIER9.bit.INTx7 = 1;
474 // ----- code for CAN end here -----
475
476 // ----- code for CAN start here -----
477 // Enable the CAN interrupt signal
478 CanbRegs.CAN_GLB_INT_EN.bit.GLBINT0_EN = 1;
479 // ----- code for CAN end here -----
480
481 // 58.5 in * 7.5 tiles = 438.75 inches = 11.14425 meters
482
483
484 robotdest[0].x = 0; robotdest[0].y = 11;
485 robotdest[1].x = -8; robotdest[1].y = 11;
486 //middle of bottom
487 // robotdest[2].x = 0; robotdest[2].y = 2;
488 // //outside the course
489 // robotdest[3].x = 0; robotdest[3].y = -3;
490 // //back to middle
491 // robotdest[4].x = 0; robotdest[4].y = 2;
492 // robotdest[5].x = 4; robotdest[5].y = 2;
493 // robotdest[6].x = 4; robotdest[6].y = 10;
494 // robotdest[7].x = 0; robotdest[7].y = 9;
495 // // the two extra points that we added for the robot to go to after the initial course one in the middle of the field and one outside near the wall - ER
496 // robotdest[8].x = 2; robotdest[8].y = 5;
497 // robotdest[9].x = -2; robotdest[9].y = -1;
498
499 // ROBOTps will be updated by Optitrack during gyro calibration
500 // TODO: specify the starting position of the robot
501 ROBOTps.x = 0; //the estimate in array form (useful for matrix operations)
502 ROBOTps.y = 0;
503 ROBOTps.theta = 0; // was -PI: need to flip OT ground plane to fix this
504 x_pred[0][0] = ROBOTps.x; //estimate in structure form (useful elsewhere)
505 x_pred[1][0] = ROBOTps.y;
506 x_pred[2][0] = ROBOTps.theta;
507
508 AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
509 PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
510 SpibRegs.SPIFFRX.bit.RXFFOVFLCLR=1; // Clear Overflow flag
511 SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
512 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
513 EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
514
515 init_serialSCIA(&SerialA,115200);
516 init_serialSCID(&SerialD,2083332);
517
518 // Enable global Interrupts and higher priority real-time debug events
519 EINT; // Enable Global interrupt INTM
520 ERTM; // Enable Global realtime interrupt DBGM
521 // ----- code for CAN start here -----
522 // Measured Distance from 1
523 // Initialize the receive message object 1 used for receiving CAN messages.
524 // Message Object Parameters:
525 // Message Object ID Number: 1
526 // Message Identifier: 0x060b0101
527 // Message Frame: Standard
528 // Message Type: Receive
529 // Message ID Mask: 0x0
530 // Message Object Flags: Receive Interrupt
531 // Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
532 // for a Receive mailbox)
533 //
534 CANsetMessageObject(CANB_BASE, RX_MSG_OBJ_ID_1, 0x060b0101, CAN_MSG_FRAME_EXT,
535 CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
536 RX_MSG_DATA_LENGTH);
537
538 // Measured Distance from 2
539 // Initialize the receive message object 2 used for receiving CAN messages.
540 // Message Object Parameters:
541 // Message Object ID Number: 2
542 // Message Identifier: 0x060b0102
543 // Message Frame: Standard
544 // Message Type: Receive
545 // Message ID Mask: 0x0
546 // Message Object Flags: Receive Interrupt
547 // Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
548 // for a Receive mailbox)
549 //
550
551 CANsetMessageObject(CANB_BASE, RX_MSG_OBJ_ID_2, 0x060b0102, CAN_MSG_FRAME_EXT,
552 CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
553 RX_MSG_DATA_LENGTH);
554
555 // Measurement Quality from 1
556 // Initialize the receive message object 2 used for receiving CAN messages.
557 // Message Object Parameters:
558 // Message Object ID Number: 3
559 // Message Identifier: 0x060b0201
560 // Message Frame: Standard
561 // Message Type: Receive
562 // Message ID Mask: 0x0
563 // Message Object Flags: Receive Interrupt
564 // Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
565 // for a Receive mailbox)
566 //

```

```

567
568 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_3, 0x060b0201, CAN_MSG_FRAME_EXT,
569                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
570                       RX_MSG_DATA_LENGTH);
571
572 // Measurement Quality from 2
573 // Initialize the receive message object 2 used for receiving CAN messages.
574 // Message Object Parameters:
575 //   Message Object ID Number: 4
576 //   Message Identifier: 0x060b0202
577 //   Message Frame: Standard
578 //   Message Type: Receive
579 //   Message ID Mask: 0x0
580 //   Message Object Flags: Receive Interrupt
581 //   Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
582 //       for a Receive mailbox)
583 //
584
585 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_4, 0x060b0202, CAN_MSG_FRAME_EXT,
586                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
587                       RX_MSG_DATA_LENGTH);
588
589 //
590 // Start CAN module operations
591 //
592 CanbRegs.CAN_CTL.bit.Init = 0;
593 CanbRegs.CAN_CTL.bit.CCE = 0;
594
595
596 // ----- code for CAN end here -----
597 char S_command[19] = "S1152000124000\n"; //this change the baud rate to 115200
598 uint16_t S_len = 19;
599 serial_sendSCIC(&SerialC, S_command, S_len);
600
601
602 DELAY_US(1000000); // Delay letting Lidar change its Baud rate
603 init_serialSCIC(&SerialC, 115200);
604
605
606 // LED1 is GPIO22
607 GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
608 // LED2 is GPIO94
609 GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
610 // LED3 is GPIO95
611 GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
612 // LED4 is GPIO97
613 GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
614 // LED5 is GPIO111
615 GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
616
617
618 // IDLE loop. Just sit and loop forever (optional):
619 while(1)
620 {
621     if (UARTPrint == 1) {
622         //UART_printfLine(1, "RCangle: %.2f", RCangle);
623         if (readbuttons() == 0) {
624             UART_printfLine(1, "RobotState: %d", RobotState);
625             // UART_printfLine(1, "01A: %.0fC: %.0fR: %.0f", MaxAreaThreshold1, MaxColThreshold1, MaxRowThreshold1);
626             // UART_printfLine(2, "PIA: %.0fC: %.0fR: %.0f", MaxAreaThreshold2, MaxColThreshold2, MaxRowThreshold2);
627             // UART_printfLine(1, "x: %.2f; y: %.2f; a: %.2f", ROBOTps.x, ROBOTps.y, ROBOTps.theta);
628             UART_printfLine(2, "orange: %.2f", robotToBall2);
629         } else if (readbuttons() == 1) {
630             // UART_printfLine(1, "01A: %.0fC: %.0fR: %.0f", MaxAreaThreshold1, MaxColThreshold1, MaxRowThreshold1);
631             // UART_printfLine(2, "PIA: %.0fC: %.0fR: %.0f", MaxAreaThreshold2, MaxColThreshold2, MaxRowThreshold2);
632             //UART_printfLine(1, "LV1: %.3f LV2: %.3f", printLV1, printLV2);
633             //UART_printfLine(2, "Ln1: %.3f Ln2: %.3f", printLinux1, printLinux2);
634         } else if (readbuttons() == 2) {
635             UART_printfLine(1, "02A: %.0fC: %.0fR: %.0f", NextLargestAreaThreshold1, NextLargestColThreshold1, NextLargestRowThreshold1);
636             UART_printfLine(2, "P2A: %.0fC: %.0fR: %.0f", NextLargestAreaThreshold2, NextLargestColThreshold2, NextLargestRowThreshold2);
637             // UART_printfLine(1, "%.2f %.2f", adcC2Volt, adcC3Volt);
638             // UART_printfLine(2, "%.2f %.2f", adcC4Volt, adcC5Volt);
639         } else if (readbuttons() == 4) {
640             UART_printfLine(1, "03A: %.0fC: %.0fR: %.0f", NextNextLargestAreaThreshold1, NextNextLargestColThreshold1, NextNextLargestRowThreshold1);
641             UART_printfLine(2, "P3A: %.0fC: %.0fR: %.0f", NextNextLargestAreaThreshold2, NextNextLargestColThreshold2, NextNextLargestRowThreshold2);
642             // UART_printfLine(1, "L: %.3f R: %.3f", LeftVel, RightVel);
643             // UART_printfLine(2, "uL: %.2f uR: %.2f", uLeft, uRight);
644         } else if (readbuttons() == 8) {
645             UART_printfLine(1, "020x%.2f y%.2f", ladar_pts[20].x, ladar_pts[20].y);
646             UART_printfLine(2, "150x%.2f y%.2f", ladar_pts[150].x, ladar_pts[150].y);
647         } else if (readbuttons() == 3) {
648             UART_printfLine(1, "Vrf: %.2f trn: %.2f", vref, turn);
649             UART_printfLine(2, "MPU: %.2f LPR: %.2f", gyro9250_radians, gyroLPR510_radians);
650         } else if (readbuttons() == 5) {
651             UART_printfLine(1, "0x: %.2f; 0y: %.2f; 0a: %.2f", OPTITRACKps.x, OPTITRACKps.y, OPTITRACKps.theta);
652             UART_printfLine(2, "State: %d : %d", RobotState, statePos);
653         } else if (readbuttons() == 6) {
654             UART_printfLine(1, "D1 %ld D2 %ld", dis_1, dis_2);
655             UART_printfLine(2, "St1 %ld St2 %ld", measure_status_1, measure_status_2);
656         } else if (readbuttons() == 7) {
657             UART_printfLine(1, "%.0f, %.1f, %.1f, %.1f", tagid, tagx, tagy, tagz);
658             UART_printfLine(2, "%.1f, %.1f, %.1f", tagthetax, tagthetay, tagthetaz);
659         }
660     }
661     UARTPrint = 0;
662 }
663 }
664 }
665
666 __interrupt void SPIB_isr(void) {
667
668     uint16_t i;
669     GpioDataRegs.GPBSET.bit.GPIO32 = 1;
670
671     GpioDataRegs.GPCSET.bit.GPIO66 = 1; //Pull CS high as done R/W
672     GpioDataRegs.GPASET.bit.GPIO29 = 1; // Pull CS high for DAN28027
673
674     if (CurrentChip == MPU9250) {
675
676         for (i=0; i<8; i++) {
677             readdata[i] = SpibRegs.SPIRXBUF; // readdata[0] is garbage
678         }
679
680         PostSWI1(); // Manually cause the interrupt for the SWI1
681
682     } else if (CurrentChip == DAN28027) {
683
684         DAN28027Garbage = SpibRegs.SPIRXBUF;
685         dan28027adc1 = SpibRegs.SPIRXBUF;
686         dan28027adc2 = SpibRegs.SPIRXBUF;
687         CurrentChip = MPU9250;
688         SpibRegs.SPIFFCT.bit.TXDLY = 0;
689     }
690
691     SpibRegs.SPIFFRX.bit.RXFFOVFLR=1; // Clear Overflow flag
692     SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
693     PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
694     GpioDataRegs.GPBCLR.bit.GPIO32 = 1;
695 }
696
697
698 //adcd1 pie interrupt
699 __interrupt void ADCC_ISR (void)
700 {
701     GpioDataRegs.GPASET.bit.GPIO11 = 1;
702     adcc2result = AdccResultRegs.ADCRESULT0;
703     adcc3result = AdccResultRegs.ADCRESULT1;
704     adcc4result = AdccResultRegs.ADCRESULT2;
705     adcc5result = AdccResultRegs.ADCRESULT3;
706
707     // Here covert ADCIND0 to volts
708     adcC2Volt = adcc2result*3.0/4095.0;

```

```

709     adcc3Volt = adcc3result*3.0/4095.0;
710     adcc4Volt = adcc4result*3.0/4095.0;
711     adcc5Volt = adcc5result*3.0/4095.0;
712
713     if (MPU9250ignoreCNT >= 1) {
714         CurrentChip = MPU9250;
715         SpibRegs.SPIFFCT.bit.TXDLY = 0;
716         SpibRegs.SPIFFRX.bit.RXFFIL = 8;
717
718         GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
719
720         SpibRegs.SPITXBUF = ((0x8000)|(0x3A00));
721         SpibRegs.SPITXBUF = 0;
722         SpibRegs.SPITXBUF = 0;
723         SpibRegs.SPITXBUF = 0;
724         SpibRegs.SPITXBUF = 0;
725         SpibRegs.SPITXBUF = 0;
726         SpibRegs.SPITXBUF = 0;
727         SpibRegs.SPITXBUF = 0;
728     } else {
729         MPU9250ignoreCNT++;
730     }
731
732     numADCCcalls++;
733     AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
734     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
735     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
736
737 }
738
739 // cpu_timer0_isr - CPU Timer0 ISR
740 __interrupt void cpu_timer0_isr(void)
741 {
742     CpuTimer0.InterruptCount++;
743
744     numTimer0calls++;
745
746     if ((numTimer0calls%5) == 0) {
747         // Blink LaunchPad Red LED
748         //GpioDataRegs.GPBTGGLE.bit.GPIO34 = 1;
749     }
750
751     // Acknowledge this interrupt to receive more interrupts from group 1
752     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
753 }
754
755 // cpu_timer1_isr - CPU Timer1 ISR
756 __interrupt void cpu_timer1_isr(void)
757 {
758     serial_sendSCIC(&SerialC, G_command, G_len);
759
760     CpuTimer1.InterruptCount++;
761
762 }
763
764 // cpu_timer2_isr CPU Timer2 ISR
765 __interrupt void cpu_timer2_isr(void)
766 {
767     // Blink LaunchPad Blue LED
768     //GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
769     PostSWI3(); //Added PostSWI3 for Lab 7 Ex.1 Step 1 - DS
770     CpuTimer2.InterruptCount++;
771
772     // if ((CpuTimer2.InterruptCount % 10) == 0) {
773     //     UARTPrint = 1;
774     // }
775 }
776
777 void setF28027EPWM1A(float controleffort){
778     if (controleffort < -10) {
779         controleffort = -10;
780     }
781     if (controleffort > 10) {
782         controleffort = 10;
783     }
784     float value = (controleffort+10)*3000.0/20.0;
785     EPwm1A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
786 }
787 void setF28027EPWM2A(float controleffort){
788     if (controleffort < -10) {
789         controleffort = -10;
790     }
791     if (controleffort > 10) {
792         controleffort = 10;
793     }
794     float value = (controleffort+10)*3000.0/20.0;
795     EPwm2A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
796 }
797
798 //
799 // Connected to PIEIER12_9 (use MINT12 and MG12_9 masks):
800 //
801 __interrupt void SWI1_HighestPriority(void) // EMIF_ERROR
802 {
803     GpioDataRegs.GPASET.bit.GPIO61 = 1;
804     // Set interrupt priority:
805     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
806     IER |= M_INT12;
807     IER &= MINT12; // Set "global" priority
808     PieCtrlRegs.PIEIER12.all &= MG12_9; // Set "group" priority
809     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
810     _asm(" NOP");
811     EINT;
812
813     //=====
814     IMU_data[0] = readdata[1];
815     IMU_data[1] = readdata[2];
816     IMU_data[2] = readdata[3];
817     IMU_data[3] = readdata[5];
818     IMU_data[4] = readdata[6];
819     IMU_data[5] = readdata[7];
820
821     accelx = (((float)(IMU_data[0]))*4.0/32767.0);
822     accely = (((float)(IMU_data[1]))*4.0/32767.0);
823     accelz = (((float)(IMU_data[2]))*4.0/32767.0);
824     gyro_x = (((float)(IMU_data[3]))*250.0/32767.0);
825     gyro_y = (((float)(IMU_data[4]))*250.0/32767.0);
826     gyro_z = (((float)(IMU_data[5]))*250.0/32767.0);
827
828     if(calibration_state == 0){
829         calibration_count++;
830         if (calibration_count == 2000) {
831             calibration_state = 1;
832             calibration_count = 0;
833         }
834     } else if(calibration_state == 1){
835         accelx_offset+=accelx;
836         accely_offset+=accely;
837         accelz_offset+=accelz;
838         gyro_x_offset+=gyro_x;
839         gyro_y_offset+=gyro_y;
840         gyro_z_offset+=gyro_z;
841         gyroLPRS10_offset+=adcc5Volt;
842         calibration_count++;
843         if (calibration_count == 2000) {
844             calibration_state = 2;
845             accelx_offset/=2000.0;
846             accely_offset/=2000.0;
847             accelz_offset/=2000.0;
848             gyro_x_offset/=2000.0;
849             gyro_y_offset/=2000.0;
850             gyro_z_offset/=2000.0;

```

```

851     gyroLPR510_offset/=2000.0;
852     calibration_count = 0;
853 }
854 } else if(calibration_state == 2) {
855
856     gyroLPR510_drift += (((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005)/(2000);
857     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
858
859     gyro9250_drift += (((gyroz-gyroz_offset) + old_gyro9250)*.0005)/(2000);
860     old_gyro9250 = gyroz-gyroz_offset;
861     calibration_count++;
862
863     if (calibration_count == 2000) {
864         calibration_state = 3;
865         calibration_count = 0;
866         doneCal = 1;
867         newOPTITRACKpose = 0;
868     }
869 } else if(calibration_state == 3){
870     accelx -=(accelx_offset);
871     accely -=(accely_offset);
872     accelz -=(accelz_offset);
873     gyrox -= gyrox_offset;
874     gyroy -= gyroy_offset;
875     gyroz -= gyroz_offset;
876     LeftWheel = readEncLeft();
877     RightWheel = readEncRight();
878     HandValue = readEncWheel();
879
880     gyro9250_angle = gyro9250_angle + (gyroz + old_gyro9250)*.0005 - gyro9250_drift;
881     old_gyro9250 = gyroz;
882
883     gyroLPR510_angle = gyroLPR510_angle + (adcC5Volt-gyroLPR510_offset) + old_gyroLPR510*.0005 - gyroLPR510_drift;
884     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
885
886     gyro9250_radians = (gyro9250_angle * (PI/180.0));
887     gyroLPR510_radians = gyroLPR510_angle * 400 * (PI/180.0);
888
889     LeftVel = (1.235/12.0)*(LeftWheel - LeftWheel_1)*1000;
890     RightVel = (1.235/12.0)*(RightWheel - RightWheel_1)*1000;
891
892     if (newLinuxCommands == 1) {
893         newLinuxCommands = 0;
894         printLinux1 = LinuxCommands[0];
895         printLinux2 = LinuxCommands[1];
896         ///? = LinuxCommands[2];
897         ///? = LinuxCommands[3];
898         ///? = LinuxCommands[4];
899         ///? = LinuxCommands[5];
900         ///? = LinuxCommands[6];
901         ///? = LinuxCommands[7];
902         ///? = LinuxCommands[8];
903         ///? = LinuxCommands[9];
904         ///? = LinuxCommands[10];
905     }
906
907     if (NewCAMDataThreshold1 == 1) {
908         NewCAMDataThreshold1 = 0;
909         MaxAreaThreshold1 = fromCAMvaluesThreshold1[0];
910         MaxColThreshold1 = fromCAMvaluesThreshold1[1];
911         MaxRowThreshold1 = fromCAMvaluesThreshold1[2];
912
913         // using the calibration values and the line of best fit found from the distance of the green golf ball in relation to the real world, this equation calculates the distance the robot is
914         robotToBall1 = -.00002154*MaxRowThreshold1*MaxRowThreshold1*MaxRowThreshold1 + 0.006486*MaxRowThreshold1*MaxRowThreshold1 - 0.6728*MaxRowThreshold1 + 25.42;
915
916         NextLargestAreaThreshold1 = fromCAMvaluesThreshold1[3];
917         NextLargestColThreshold1 = fromCAMvaluesThreshold1[4];
918         NextLargestRowThreshold1 = fromCAMvaluesThreshold1[5];
919
920         NextNextLargestAreaThreshold1 = fromCAMvaluesThreshold1[6];
921         NextNextLargestColThreshold1 = fromCAMvaluesThreshold1[7];
922         NextNextLargestRowThreshold1 = fromCAMvaluesThreshold1[8];
923         numThres1++;
924         if ((numThres1 % 5) == 0) {
925             // LED4 is GPIO97
926             GpioDataRegs.GPDTOGGLE.bit.GPIO97 = 1;
927         }
928     }
929
930     if (NewCAMDataThreshold2 == 1) {
931         NewCAMDataThreshold2 = 0;
932         MaxAreaThreshold2 = fromCAMvaluesThreshold2[0];
933         MaxColThreshold2 = fromCAMvaluesThreshold2[1];
934         MaxRowThreshold2 = fromCAMvaluesThreshold2[2];
935         // using the calibration values and the line of best fit found from the distance of the green golf ball in relation to the real world, this equation calculates the distance the robot is
936         robotToBall2 = -.00002154*MaxRowThreshold2*MaxRowThreshold2*MaxRowThreshold2 + 0.006486*MaxRowThreshold2*MaxRowThreshold2 - 0.6728*MaxRowThreshold2 + 25.42;
937
938         NextLargestAreaThreshold2 = fromCAMvaluesThreshold2[3];
939         NextLargestColThreshold2 = fromCAMvaluesThreshold2[4];
940         NextLargestRowThreshold2 = fromCAMvaluesThreshold2[5];
941
942         NextNextLargestAreaThreshold2 = fromCAMvaluesThreshold2[6];
943         NextNextLargestColThreshold2 = fromCAMvaluesThreshold2[7];
944         NextNextLargestRowThreshold2 = fromCAMvaluesThreshold2[8];
945         numThres2++;
946         if ((numThres2 % 5) == 0) {
947             // LED5 is GPIO111
948             GpioDataRegs.GPDTOGGLE.bit.GPIO111 = 1;
949         }
950     }
951
952     if (NewCAMDataAprilTag1 == 1) {
953         NewCAMDataAprilTag1 = 0;
954         tagx = fromCAMvaluesAprilTag1[0];
955         tagy = fromCAMvaluesAprilTag1[1];
956         tagz = fromCAMvaluesAprilTag1[2];
957
958         tagthetax = fromCAMvaluesAprilTag1[3];
959         tagthetay = fromCAMvaluesAprilTag1[4];
960         tagthetaz = fromCAMvaluesAprilTag1[5];
961
962         tagid = fromCAMvaluesAprilTag1[6];
963
964         numtag++;
965         if ((numtag % 5) == 0) {
966             // LED2 is GPIO94
967             GpioDataRegs.GPDTOGGLE.bit.GPIO94 = 1;
968         }
969     }
970
971     if (NewLVData == 1) {
972         NewLVData = 0;
973         printLV1 = fromLVvalues[0];
974         printLV2 = fromLVvalues[1];
975         ///? = fromLVvalues[2];
976         ///? = fromLVvalues[3];
977         ///? = fromLVvalues[4];
978         ///? = fromLVvalues[5];
979         ///? = fromLVvalues[6];
980         ///? = fromLVvalues[7];
981     }
982
983
984     if (new_optitrack == 1) {
985         OPTITRACKps = UpdateOptitrackStates(ROBOTps, &newOPTITRACKpose);
986         new_optitrack = 0;
987     }
988
989     // Step 0: update B, u
990     B[0][0] = cosf(ROBOTps.theta)*0.001;
991     B[1][0] = sinf(ROBOTps.theta)*0.001;
992     B[2][1] = 0.001;

```



```

993 u[0][0] = 0.5*(LeftVel + RightVel); // linear velocity of robot
994 u[1][0] = gyrozz*(PI/180.0); // angular velocity in rad/s (negative for right hand angle)
995
996 // Step 1: predict the state and estimate covariance
997 Matrix3x2 Mult(B, u, Bu); // Bu = B*u
998 Matrix3x1 Add(x_pred, Bu, x_pred, 1.0, 1.0); // x_pred = x_pred(old) + Bu
999 Matrix3x3 Add(P_pred, 0, P_pred, 1.0, 1.0); // P_pred = P_pred(old) + Q
1000 // Step 2: if there is a new measurement, then update the state
1001 if (1 == newOPTITRACKpose) {
1002     OptiNumUpdatesProcessed++; // checking how often OptiTrack is causing a Kalman update
1003     if ((OptiNumUpdatesProcessed%10)==0) {
1004         // LED 3
1005         GpioDataRegs.GPCTOGGLE.bit.GPIO95 = 1;
1006     }
1007     newOPTITRACKpose = 0;
1008     z[0][0] = OPTITRACKps.x; // take in the Optitrack Motion Capture measurement
1009     z[1][0] = OPTITRACKps.y;
1010     // fix for OptiTrack problem at 180 degrees
1011     if (cosf(ROBOTps.theta) < -0.99) {
1012         z[2][0] = ROBOTps.theta;
1013     }
1014     else {
1015         z[2][0] = OPTITRACKps.theta;
1016     }
1017     // Step 2a: calculate the innovation/measurement residual, ytilde
1018     Matrix3x1 Add(z, x_pred, ytilde, 1.0, -1.0); // ytilde = z-x_pred
1019     // Step 2b: calculate innovation covariance, S
1020     Matrix3x3 Add(P_pred, R, S, 1.0, 1.0); // S = P_pred + R
1021     // Step 2c: calculate the optimal Kalman gain, K
1022     Matrix3x3 Invert(S, S_inv);
1023     Matrix3x3 Mult(P_pred, S_inv, K); // K = P_pred*(S^-1)
1024     // Step 2d: update the state estimate x_pred = x_pred(old) + K*ytilde
1025     Matrix3x1 Mult(K, ytilde, temp_3x1);
1026     Matrix3x1 Add(x_pred, temp_3x1, x_pred, 1.0, 1.0);
1027     // Step 2e: update the covariance estimate P_pred = (I-K)*P_pred(old)
1028     Matrix3x3 Add(eye3, K, temp_3x3, 1.0, -1.0);
1029     Matrix3x3 Mult(temp_3x3, P_pred, P_pred);
1030 } // end of correction step
1031
1032 // set ROBOTps to the updated and corrected Kalman values.
1033 ROBOTps.x = x_pred[0][0];
1034 ROBOTps.y = x_pred[1][0];
1035 ROBOTps.theta = x_pred[2][0];
1036
1037 // Make sure this function is called every time in this function even if you decide not to use its vref and turn
1038 // uses xy code to step through an array of positions
1039 if( xy_control(&vref, &turn, 1.0, ROBOTps.x, ROBOTps.y, robotdest[statePos].x, robotdest[statePos].y, ROBOTps.theta, 0.25, 0.5)) {
1040     statePos = (statePos+1)%NUMWAYPOINTS;
1041 }
1042
1043 dwell++;
1044 // state machine
1045 colcentroid1 = MaxColThreshold1 - 80;
1046 colcentroid2 = MaxColThreshold2 - 80; // 80 is used as the center column of the camera, so a centered green or orange ball centered gives colcentroid = 0 - DS
1047 switch (RobotState) {
1048 case 1: // command the robot to an X,Y point in the course - KL
1049     // vref and turn are the vref and turn returned from xy_control
1050     if (LADARfront < 1.2) {
1051         vref = 0.2;
1052         checkfronttally++;
1053         if (checkfronttally > 310) { // check if LADARfront < 1.2 for 310ms or 3 LADAR samples
1054             RobotState = 10; // Wall follow
1055             WallFollowtime = 0;
1056             right_wall_follow_state = 1;
1057         }
1058     } else {
1059         checkfronttally = 0;
1060     }
1061     // ensures that the robot does not move for 2 seconds after returning to state 1, so after is stops detecting a golf ball - ER
1062     statecount++;
1063     if (statecount<=2000){
1064         RobotState = 1;
1065     }else{
1066         if (MaxAreaThreshold1 > 30) { // green ball detection threshold chosen so camera noise (image disturbance, not literal) does not trigger chasing rolling ball behavior - DS
1067             RobotState = 20;
1068             statecount = 0;
1069         }
1070         if (MaxAreaThreshold2 > 30) { // same detection threshold but for orange ball - DS
1071             RobotState = 30;
1072             statecount = 0;
1073         }
1074         if (tagx <= 0.0993) {
1075             if (tagid == 0.0){
1076                 RobotState = 40;
1077             }
1078             if (tagid == 1.0){
1079                 RobotState = 50;
1080             }
1081         }
1082     }
1083 }
1084 break;
1085 case 10: // right wall following if an obstacle gets in the way of robot - KL
1086     if (right_wall_follow_state == 1) {
1087         //Left Turn
1088         turn = Kp_front_wall*(14.5 - LADARfront);
1089         vref = front_turn_velocity;
1090         if (LADARfront > left_turn_Stop_threshold) {
1091             right_wall_follow_state = 2;
1092         }
1093     } else if (right_wall_follow_state == 2) {
1094         //Right Wall Follow
1095         turn = Kp_right_wall*(ref_right_wall - LADARrightfront);
1096         vref = foward_velocity;
1097         if (LADARfront < left_turn_Start_threshold) {
1098             right_wall_follow_state = 1;
1099         }
1100     }
1101     if (turn > turn_saturation) {
1102         turn = turn_saturation;
1103     }
1104     if (turn < -turn_saturation) {
1105         turn = -turn_saturation;
1106     }
1107
1108     WallFollowtime++;
1109     // exit wall following if at least 5 seconds have passed and front path is clear - KL
1110     if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
1111         RobotState = 1; //return to XY waypoint navigation - KL
1112         checkfronttally = 0;
1113     }
1114     break;
1115 // this state sets vref and turn so that the robot moves towards a detected green golf ball and after 1 second of waiting, sends the robot to state 22 which stops the robot - ER
1116 case 20:
1117     // put vision code here
1118     if (MaxColThreshold1 == 0 || MaxAreaThreshold1 < 3){
1119         vref = 0;
1120         turn = 0;
1121     } else {
1122         vref = 0.75;
1123         turn = kpvision * (0 - colcentroid1);
1124         if (MaxRowThreshold1 > 100){ //KLEC: row number for rb to know how to close it is to the ball, need to tune the number for enough clearance so that gate doesn't push ball away
1125             RobotState = 22;
1126             count = 0;
1127         }
1128     }
1129     break;
1130 case 22: // after no longer detecting the green golf ball, the robot stops moving - ER, lift arm and move tongue
1131     vref = 0;
1132     turn = 0;
1133
1134     count++;

```



```

1135         if (count>=1000){//after 1 second, robot drives forward - KL
1136             RobotState = 24;
1137             count = 0;
1138         }
1139         break;
1140     case 24: // robot drives forward into the golf ball - KL
1141         vref = 0.5;
1142         turn = 0;
1143         setEPWM5B_RCServo(LTong); //KLEC: move the tongue to the left side, green balls go to the right compartment
1144         setEPWM6A_RCServo(Gate_0); //KLEC: open the gate
1145         count++;
1146         if (count>=2000){
1147             RobotState = 26;
1148             count = 0;
1149         }
1150         break;
1151     case 26: // robot pauses once ball is presumed to be captured - KL
1152         vref = 0;
1153         turn = 0;
1154         setEPWM6A_RCServo(Gate_C);
1155         setEPWM5B_RCServo(MTong);
1156         count++;
1157         if (count>=1000){ //after 1 second, robot returns to waypoint navigation - KL
1158             RobotState = 1;
1159             count = 0;
1160             statecount = 0;
1161         }
1162         break;
1163
1164     // states for orange ball
1165     case 30:
1166         // put vision code here
1167         //saying that if the orange ball is not detected or doesn't fit in the area threshold, then to stop the robot - KP
1168         //this helps it not detect other orange objects in the room as the ball - KP
1169         if (MaxColThreshold2 == 0 || MaxAreaThreshold2 < 3){
1170             vref = 0;
1171             turn = 0;
1172         } else {
1173             //tells it what to do if it is detected, move to state 32 - KP
1174             vref = 0.75;
1175             turn = kpvision * (0 - colcentroid2);
1176             if (MaxRowThreshold2 > 100){
1177                 RobotState = 32;
1178                 count = 0;
1179             }
1180         }
1181         break;
1182     case 32:
1183         //stopping and pausing with vref and turn at 0 - KP
1184         vref = 0;
1185         turn = 0;
1186         setEPWM5B_RCServo(RTong); //KLEC: move the tongue to the right side, orange balls go to the left compartment
1187         setEPWM6A_RCServo(Gate_0); //KLEC: open the gate
1188         count++;
1189         if (count>=1000){
1190             RobotState = 34;
1191             count = 0;
1192         }
1193         break;
1194     case 34:
1195         //telling robot to move forward then move to state 36 - KP
1196         vref = 0.5;
1197         turn = 0;
1198         setEPWM5B_RCServo(RTong);
1199         setEPWM6A_RCServo(Gate_0);
1200         count++;
1201         if (count>=1000){
1202             RobotState = 36;
1203             count = 0;
1204         }
1205         break;
1206     case 36:
1207         //another stop before it begins the route again - KP
1208         vref = 0;
1209         turn = 0;
1210         setEPWM6A_RCServo(Gate_C);
1211         setEPWM5B_RCServo(MTong);
1212         count++;
1213         if (count>=1000){
1214             RobotState = 1;
1215             count = 0;
1216             statecount = 0;
1217         }
1218         break;
1219     case 40: //code telling the gate to open when it detects april tag 0
1220         vref = 0;
1221         turn = 0;
1222
1223         setEPWM6A_RCServo(Gate_0);
1224         setEPWM5B_RCServo(0);
1225
1226         if (dwell >= 2000){
1227             setEPWM6A_RCServo(Gate_C);
1228         }
1229
1230         count++;
1231         if (count>=1000){
1232             RobotState = 1;
1233             count = 0;
1234             statecount = 0;
1235         }
1236         break;
1237     case 50: //code telling the robot to turn
1238
1239         vref = 0;
1240         count++;
1241         if (count>=2000){
1242             RobotState = 52;
1243             count = 0;
1244         }
1245         break;
1246     case 52:
1247         current = ROBOTps.theta;
1248         target = PI;
1249         turn = fabs(target - current);
1250         vref = 1;
1251         count++;
1252         if (count>=1000){
1253             RobotState = 1;
1254             count = 0;
1255             statecount = 0;
1256         }
1257         break;
1258
1259     default:
1260         break;
1261 }
1262
1263
1264 //Must be called each time into this SWI1 function
1265 PIcontrol(&uLeft,&uRight,vref,turn,LeftWheel,RightWheel);
1266 // These below lines also must be called each time into this SWI1 function
1267 setEPWM1A(uLeft);
1268 setEPWM2A(-uRight);
1269 setF28027EPWM1A(uLeft);
1270 setF28027EPWM2A(-uRight);
1271 LeftWheel_1 = LeftWheel;
1272 RightWheel_1 = RightWheel;
1273
1274 if((timecount%250) == 0) {
1275     DataToLabView.floatData[0] = ROBOTps.x;
1276     DataToLabView.floatData[1] = ROBOTps.y;

```

```

1277     DataToLabView.floatData[2] = ROBOTps.theta;
1278     DataToLabView.floatData[3] = (float)timecount;
1279     DataToLabView.floatData[4] = (float)(LeftVel + RightVel);
1280     DataToLabView.floatData[5] = (float)RobotState;
1281     DataToLabView.floatData[6] = (float)statePos;
1282     DataToLabView.floatData[7] = LADARfront;
1283     LVsenddata[0] = '*'; // header for LVdata
1284     LVsenddata[1] = '$';
1285     for (i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
1286         if (i%2==0) {
1287             LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
1288         } else {
1289             LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;
1290         }
1291     }
1292     serial_sendSCID(&SerialD, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
1293 }
1294
1295 }
1296 timecount++;
1297 if((timecount%200) == 0)
1298 {
1299     if(doneCal == 0) {
1300         GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1; // Blink Blue LED while calibrating
1301     }
1302     GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1; // Always Block Red LED
1303     UARTPrint = 1; // Tell while loop to print
1304 }
1305
1306 SpibRegs.SPIFFCT.bit.TXDLY = 16;
1307 CurrentChip = DAN28027;
1308 SpibRegs.SPIFFRX.bit.RXFFIL = 3;
1309 GpioDataRegs.GPCLEAR.bit.GPIO29 = 1;
1310 SpibRegs.SPITXBUF = 0x00DA;
1311 SpibRegs.SPITXBUF = EPwm1A_F28027;
1312 SpibRegs.SPITXBUF = EPwm2A_F28027;
1313
1314 //*****
1315 //
1316 // Restore registers saved:
1317 //
1318 DINT;
1319 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1320
1321 GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
1322 }
1323
1324 //
1325 // Connected to PIEIER12_10 (use MINT12 and MG12_10 masks):
1326 //
1327 __interrupt void SWI2_MiddlePriority(void) // RAM_CORRECTABLE_ERROR
1328 {
1329     // Set interrupt priority:
1330     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1331     IER |= M_INT12;
1332     IER &= MINT12; // Set "global" priority
1333     PieCtrlRegs.PIEIER12.all &= MG12_10; // Set "group" priority
1334     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1335     asm(" NOP");
1336     EINT;
1337
1338 //*****
1339 // Insert SWI ISR Code here.....
1340 // LED1
1341 GpioDataRegs.GPATOGGLE.bit.GPIO22 = 1;
1342 if (LADARpingpong == 1) {
1343     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1344     LADARrightfront = 19; // 19 is greater than max feet
1345     for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1346         if (ladar_data[LADARi].distance_ping < LADARrightfront) {
1347             LADARrightfront = ladar_data[LADARi].distance_ping;
1348         }
1349     }
1350     // LADARfront is the min of dist 111, 112, 113, 114, 115
1351     LADARfront = 19;
1352     for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1353         if (ladar_data[LADARi].distance_ping < LADARfront) {
1354             LADARfront = ladar_data[LADARi].distance_ping;
1355         }
1356     }
1357     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1358     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1359     for (LADARi = 0; LADARi < 228; LADARi++) {
1360
1361         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1362         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1363     }
1364 } else if (LADARpingpong == 0) {
1365     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1366     LADARrightfront = 19; // 19 is greater than max feet
1367     for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1368         if (ladar_data[LADARi].distance_pong < LADARrightfront) {
1369             LADARrightfront = ladar_data[LADARi].distance_pong;
1370         }
1371     }
1372     // LADARfront is the min of dist 111, 112, 113, 114, 115
1373     LADARfront = 19;
1374     for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1375         if (ladar_data[LADARi].distance_pong < LADARfront) {
1376             LADARfront = ladar_data[LADARi].distance_pong;
1377         }
1378     }
1379     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1380     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1381     for (LADARi = 0; LADARi < 228; LADARi++) {
1382
1383         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_pong*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1384         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_pong*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1385     }
1386 }
1387 }
1388 }
1389
1390 //*****
1391 //
1392 // Restore registers saved:
1393 //
1394 DINT;
1395 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1396
1397 }
1398
1399 //
1400 // Connected to PIEIER12_11 (use MINT12 and MG12_11 masks):
1401 //
1402 __interrupt void SWI3_LowestPriority(void) // FLASH_CORRECTABLE_ERROR
1403 {
1404     // Set interrupt priority:
1405     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1406     IER |= M_INT12;
1407     IER &= MINT12; // Set "global" priority
1408     PieCtrlRegs.PIEIER12.all &= MG12_11; // Set "group" priority
1409     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1410     asm(" NOP");
1411     EINT;
1412     RCangle = readEncWheel(); //Lab RC servo exercise uses encoder angle as the command input for servo motion - DS
1413     // setEPWM3A_RCServo(RCangle); //RCangle is a value from -90 to 90
1414     // setEPWM3B_RCServo(RCangle); //RCangle is a value from -90 to 90
1415     // setEPWM5A_RCServo(RCangle); //RCangle is a value from -90 to 90 in use
1416     setEPWM5B_RCServo(RCangle); //RCangle is a value from -90 to 90 in use - Tongue
1417     setEPWM6A_RCServo(RCangle); //RCangle is a value from -90 to 90 in use - Gate

```

```

1419 //#####
1420 // Insert SWI ISR Code here.....
1421
1422 //#####
1423 //
1424 // Restore registers saved:
1425 //
1426 DINT;
1427 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1428
1429 }
1430
1431 // ----- code for CAN start here -----
1432 _interrupt void can_isr(void)
1433 {
1434     int i = 0;
1435
1436     uint32_t status;
1437
1438     //
1439     // Read the CAN interrupt status to find the cause of the interrupt
1440     //
1441     status = CANGetInterruptCause(CANB_BASE);
1442
1443     //
1444     // If the cause is a controller status interrupt, then get the status
1445     //
1446     if(status == CAN_INT_INT0ID_STATUS)
1447     {
1448         //
1449         // Read the controller status. This will return a field of status
1450         // error bits that can indicate various errors. Error processing
1451         // is not done in this example for simplicity. Refer to the
1452         // API documentation for details about the error status bits.
1453         // The act of reading this status will clear the interrupt.
1454         //
1455         status = CANGetStatus(CANB_BASE);
1456
1457     }
1458
1459     //
1460     // Check if the cause is the receive message object 2
1461     //
1462     else if(status == RX_MSG_OBJ_ID_1)
1463     {
1464         //
1465         // Get the received message
1466         //
1467         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_1, rxMsgData);
1468
1469         for(i = 0; i<2; i++)
1470         {
1471             dis_raw_1[i] = rxMsgData[i];
1472         }
1473
1474         dis_1 = 256*dis_raw_1[1] + dis_raw_1[0];
1475
1476         measure_status_1 = rxMsgData[2];
1477
1478         //
1479         // Getting to this point means that the RX interrupt occurred on
1480         // message object 2, and the message RX is complete. Clear the
1481         // message object interrupt.
1482         //
1483         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_1);
1484
1485         //
1486         // Increment a counter to keep track of how many messages have been
1487         // received. In a real application this could be used to set flags to
1488         // indicate when a message is received.
1489         //
1490         rxMsgCount_1++;
1491
1492         //
1493         // Since the message was received, clear any error flags.
1494         //
1495         errorFlag = 0;
1496     }
1497
1498     else if(status == RX_MSG_OBJ_ID_2)
1499     {
1500         //
1501         // Get the received message
1502         //
1503         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_2, rxMsgData);
1504
1505         for(i = 0; i<2; i++)
1506         {
1507             dis_raw_2[i] = rxMsgData[i];
1508         }
1509
1510         dis_2 = 256*dis_raw_2[1] + dis_raw_2[0];
1511
1512         measure_status_2 = rxMsgData[2];
1513
1514         //
1515         // Getting to this point means that the RX interrupt occurred on
1516         // message object 2, and the message RX is complete. Clear the
1517         // message object interrupt.
1518         //
1519         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_2);
1520
1521         //
1522         // Increment a counter to keep track of how many messages have been
1523         // received. In a real application this could be used to set flags to
1524         // indicate when a message is received.
1525         //
1526         rxMsgCount_2++;
1527
1528         //
1529         // Since the message was received, clear any error flags.
1530         //
1531         errorFlag = 0;
1532     }
1533
1534     else if(status == RX_MSG_OBJ_ID_3)
1535     {
1536         //
1537         // Get the received message
1538         //
1539         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_3, rxMsgData);
1540
1541         for(i = 0; i<4; i++)
1542         {
1543             lightlevel_raw_1[i] = rxMsgData[i];
1544             quality_raw_1[i] = rxMsgData[i+4];
1545         }
1546
1547         lightlevel_1 = ((256.0*256.0*256.0)*lightlevel_raw_1[3] + (256.0*256.0)*lightlevel_raw_1[2] + 256.0*lightlevel_raw_1[1] + lightlevel_raw_1[0])/65535;
1548         quality_1 = ((256.0*256.0*256.0)*quality_raw_1[3] + (256.0*256.0)*quality_raw_1[2] + 256.0*quality_raw_1[1] + quality_raw_1[0])/65535;
1549
1550
1551         //
1552         // Getting to this point means that the RX interrupt occurred on
1553         // message object 2, and the message RX is complete. Clear the
1554         // message object interrupt.
1555         //
1556         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_3);
1557
1558         //
1559         // Since the message was received, clear any error flags.
1560

```

```

1561 //
1562     errorFlag = 0;
1563 }
1564
1565
1566 else if(status == RX_MSG_OBJ_ID_4)
1567 {
1568     //
1569     // Get the received message
1570     //
1571     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_4, rxMsgData);
1572
1573     for(i = 0; i<4; i++)
1574     {
1575         lightlevel_raw_2[i] = rxMsgData[i];
1576         quality_raw_2[i] = rxMsgData[i+4];
1577     }
1578
1579     lightlevel_2 = ((256.0*256.0*256.0)*lightlevel_raw_2[3] + (256.0*256.0)*lightlevel_raw_2[2] + 256.0*lightlevel_raw_2[1] + lightlevel_raw_2[0])/65535;
1580     quality_2 = ((256.0*256.0*256.0)*quality_raw_2[3] + (256.0*256.0)*quality_raw_2[2] + 256.0*quality_raw_2[1] + quality_raw_2[0])/65535;
1581
1582     //
1583     // Getting to this point means that the RX interrupt occurred on
1584     // message object 2, and the message RX is complete. Clear the
1585     // message object interrupt.
1586     //
1587     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_4);
1588
1589     //
1590     // Since the message was received, clear any error flags.
1591     //
1592     errorFlag = 0;
1593 }
1594
1595
1596
1597
1598 //
1599 // If something unexpected caused the interrupt, this would handle it.
1600 //
1601 else
1602 {
1603     //
1604     // Spurious interrupt handling can go here.
1605     //
1606 }
1607
1608 //
1609 // Clear the global interrupt flag for the CAN interrupt line
1610 //
1611 CANclearGlobalInterruptStatus(CANB_BASE, CAN_GLOBAL_INT_CANINT0);
1612
1613 //
1614 // Acknowledge this interrupt located in group 9
1615 //
1616 InterruptclearACKGroup(INTERRUPT_ACK_GROUP9);
1617 }
1618 // ----- code for CAN end here -----

```